

TP1 NoSQL : Sécurité MongoDB

Authentification Déploiement Sécurisé - Partie 1

Travail Pratique N°1 - Module NoSQL



Saad KORCHI

Étudiant en Cybersécurité

Encadré par :

Pr. AMAMOU Ahmed

EIDIA Cybersécurité

SOMMAIRE

1	Introduction	3
2	Architecture NoSQL du projet	3
2.1	Pourquoi MongoDB pour un SOC?	3
2.2	Environnement d'exécution	3
3	Déploiement sécurisé de MongoDB	4
3.1	Commande Docker	4
3.2	Explication détaillée des paramètres de sécurité	4
3.3	Bonnes pratiques de sécurité supplémentaires	4
4	Authentification MongoDB	4
4.1	Tentative de connexion sans authentification	5
4.2	Connexion correcte avec authentification	5
4.3	Explication des paramètres de connexion	5
4.4	Principes fondamentaux de l'authentification MongoDB	5
	Captures d'écran	6

1 Introduction

Contexte du projet SOC-lite

Dans le cadre du projet **SOC-lite**, une base de données NoSQL a été utilisée pour stocker les logs de sécurité et les incidents. **MongoDB** a été choisi pour sa flexibilité et sa capacité à gérer des données semi-structurées. Cependant, par défaut, MongoDB n'impose pas de mécanismes de sécurité stricts.

Cette première partie du projet consiste à :

- Comprendre l'architecture NoSQL du projet
- Déployer MongoDB de manière sécurisée avec Docker
- Mettre en place l'authentification MongoDB

La sécurisation des bases de données NoSQL est un enjeu critique dans les architectures modernes. Contrairement aux bases de données relationnelles qui bénéficient de décennies d'évolution en matière de sécurité, les bases NoSQL comme MongoDB ont souvent été déployées sans mesures de protection adéquates, ce qui a conduit à de nombreuses fuites de données par le passé. Ce TP vise à combler ces lacunes en implémentant une stratégie de sécurité complète.

2 Architecture NoSQL du projet

La base de données utilisée est **MongoDB** avec une base nommée `soc_lite`. Les principales collections sont :

- **security_logs** : stocke les événements de sécurité (tentatives de connexion, alertes, actions suspectes)
- **incidents** : stocke les incidents détectés par le SOC (attaques, vulnérabilités, compromissions)
- **users** : stocke les utilisateurs de l'application SOC-lite

2.1 Pourquoi MongoDB pour un SOC ?

Le choix de MongoDB pour un SOC (Security Operations Center) présente plusieurs avantages :

- **Schéma flexible** : Les logs de sécurité ont des structures variables selon les sources
- **Performances en écriture** : Capacité à ingérer un grand volume de logs en temps réel
- **Indexation puissante** : Recherche rapide dans les incidents et alertes
- **Scalabilité horizontale** : Possibilité de distribuer la charge sur plusieurs serveurs

2.2 Environnement d'exécution

MongoDB est exécuté via **Docker** afin d'assurer un environnement isolé, reproductible et facilement déployable. Cette approche de conteneurisation permet d'éviter les conflits

de dépendances et de garantir que l'environnement de développement est identique à l'environnement de production.

3 Déploiement sécurisé de MongoDB

MongoDB a été lancé avec authentification activée en définissant un utilisateur administrateur. Cette étape est fondamentale car, par défaut, MongoDB s'exécute sans authentification, ce qui représente une vulnérabilité critique.

3.1 Commande Docker

```
1 docker run -d --name mongo_secure \  
2   -p 127.0.0.1:27017:27017 \  
3   -e MONGO_INITDB_ROOT_USERNAME=adminSoc \  
4   -e MONGO_INITDB_ROOT_PASSWORD=Passw0rd_S3cure! \  
5   mongo:7
```

3.2 Explication détaillée des paramètres de sécurité

- `-p 127.0.0.1:27017:27017` : Accès limité à localhost uniquement. MongoDB n'est pas exposé sur le réseau externe, ce qui empêche les attaques distantes directes. Seul le backend s'exécutant sur la même machine peut se connecter.
- `-e MONGO_INITDB_ROOT_USERNAME=adminSoc` : Création d'un utilisateur root personnalisé. L'utilisation d'un nom d'utilisateur non standard augmente la sécurité par obscurité et évite les attaques par dictionnaire ciblant les comptes par défaut.
- `-e MONGO_INITDB_ROOT_PASSWORD=Passw0rd_S3cure!` : Définition d'un mot de passe robuste respectant les critères de complexité (majuscules, minuscules, chiffres, caractères spéciaux).
- `mongo:7` : Utilisation de la dernière version stable de MongoDB, bénéficiant des derniers correctifs de sécurité.

3.3 Bonnes pratiques de sécurité supplémentaires

En complément de cette configuration, il est recommandé de :

- Utiliser des volumes Docker pour persister les données de manière sécurisée
- Configurer des sauvegardes automatisées de la base de données
- Mettre en place une surveillance des tentatives de connexion échouées

4 Authentification MongoDB

L'authentification est le premier rempart contre les accès non autorisés. Sans authentification, n'importe quel client capable d'atteindre le serveur MongoDB peut lire, modifier ou supprimer toutes les données.

4.1 Tentative de connexion sans authentification

```
1 mongosh
2 show dbs
```

Résultat : La commande échoue car l'accès à la base est refusé. MongoDB retourne une erreur indiquant que l'authentification est requise. Ce comportement est souhaitable car il bloque toute tentative d'accès non authentifié.

4.2 Connexion correcte avec authentification

```
1 mongosh -u adminSoc -p Passw0rd_S3cure! --authenticationDatabase
  admin
```

4.3 Explication des paramètres de connexion

- `-u adminSoc` : Spécifie le nom d'utilisateur créé lors du déploiement
- `-p Passw0rd_S3cure!` : Fournit le mot de passe associé
- `-authenticationDatabase admin` : Indique que les identifiants sont stockés dans la base `admin`, qui est la base d'authentification par défaut pour les utilisateurs root

4.4 Principes fondamentaux de l'authentification MongoDB

MongoDB utilise un mécanisme d'authentification par défi-réponse (SCRAM - Salted Challenge Response Authentication Mechanism). Ce protocole :

- Ne transmet jamais le mot de passe en clair sur le réseau
- Utilise un sel cryptographique pour protéger contre les attaques par rainbow tables
- Supporte différents mécanismes selon les versions (SCRAM-SHA-1, SCRAM-SHA-256)

Fin du TP1 NoSQL - La suite sera traitée dans le TP2

Captures d'écran

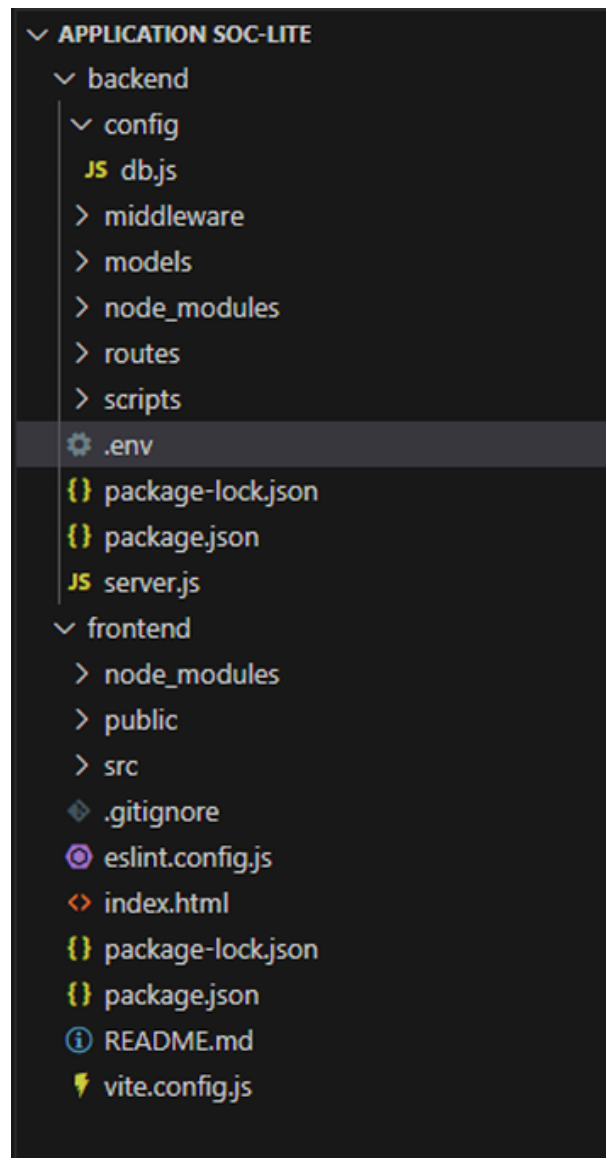


FIGURE 1 – Figure 1

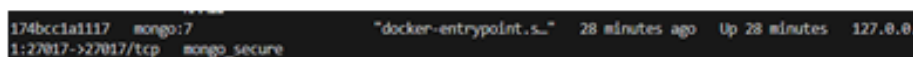


FIGURE 2 – Figure 2

```

PS D:\4eme-année\application SOC-lite> docker inspect mongo_secure
[
  {
    "Id": "174bcc1a1117125715386dc2746ec5972a74d1c91786bd199ff680a9770630fb",
    "Created": "2026-04-05T12:46:36.027517643Z",
    "Path": "docker-entrypoint.sh",
    "Args": [
      "mongod"
    ],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
    }
  }
]

```

FIGURE 3 – Figure 3

```

PS D:\4eme-année\application SOC-lite> docker exec -it mongo_secure mongosh
Current Mongosh Log ID: 69d26196d8e5d084c58563b0
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:      7.0.30
Using Mongosh:      2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

test> show dbs
MongoServerError[Unauthorized]: Command listDatabases requires authentication
test> 

```

FIGURE 4 – Figure 4

```

PS D:\4eme-année\application SOC-lite> docker exec -it mongo_secure mongosh -u adminSoc -p Passw0rd_S3cure! --authenticationDatabase admin
Current Mongosh Log ID: 69d1b5a3b4ec7d38758563b0
Connecting to:      mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&authSource=admin&appName=mongosh+2.7.0
Using MongoDB:      7.0.30
Using Mongosh:      2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-04-05T00:53:28.877+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
-----

test> show dbs
admin      100.00 KiB
config     100.00 KiB
local      72.00 KiB
soc_lite   300.00 KiB

```

FIGURE 5 – Figure 5